Registro de decisões

Convenções do contrato

Revisamos o CONTRATO_API.md e fechamos as convenções abaixo. Para cada uma está o motivo e a opção que descartamos. O critério foi o mesmo em todas: a escolha que atende o requisito com menos código e menos chance de erro dos módulos seguintes.

1. IDs numéricos (Long)

Por quê: o DER aprovado na issue #2 usa bigint. Mudar agora obrigaria a refazer o modelo e as migrações.

Descartado: UUID. Esconde a sequência dos registros, mas exige alterar o DER e deixa as URLs maiores. Não há requisito que peça isso.

2. Quantidades como número JSON decimal, até 3 casas

Por quê: o banco é numeric(12,3), com no máximo 9 dígitos inteiros, e valores assim cabem sem perda de precisão no JavaScript. Número é o formato mais simples para a interface e para o Postman, e no Java continua BigDecimal (RN02).

Descartado: texto (“2.500”). Evita qualquer risco de arredondamento, mas a interface teria de converter em toda leitura e escrita.

3. Datas ISO 8601 com offset (OffsetDateTime)

Por quê: o RNF05 exige fuso ou offset, e o agendamento futuro (RF10) compara instantes. É o formato que o Spring já serializa sem configuração extra.

Descartado: sempre UTC (“Z”). Funciona, mas a tela teria de converter e o morador veria horas diferentes das digitadas. Também descartado: tempo em milissegundos, ilegível nos testes e nos exemplos.

4. Enums em maiúsculas sem acento

Por quê: combina com @Enumerated(EnumType.STRING) e com o CHECK do banco (decisão 2.11 da issue #2). A interface mostra “Concluída” a partir de CONCLUIDA.

Descartado: usar “CONCLUÍDA” com acento, que dá problema de codificação em enum, banco e URL.

5. Versão no corpo (campo versao)

Por quê: o contrato já previa assim e a interface envia a versão que recebeu junto com os demais dados da ação. A versão diferente retorna 409 CONFLITO_VERSAO (RN06).

Descartado: cabeçalho If-Match com ETag. É o padrão HTTP, mas exige tratar cabeçalhos em cada controller e em cada chamada do frontend.

6. ID inexistente no corpo retorna 404; incompatível retorna 409

Por quê: uma regra única, fácil de lembrar e de testar: não existe é 404, existe mas não serve é 409, malformado é 400.

Descartado: 400 para ID inexistente no corpo. Mistura erro de formato com erro de dado e deixa a interface sem saber se o morador digitou errado ou o registro foi removido.

Erros

7. Formato único com código, mensagem e errosCampos

Por quê: é o que a issue e o RNF03 pedem. errosCampos é sempre uma lista (vazia em erro de regra), então o frontend não precisa testar se o campo existe. O campo usa o caminho do JSON (itens[0].quantidadeEstimada), que é o que a tela precisa para marcar o campo certo.

Descartado: ProblemDetail (RFC 9457), que o Spring oferece pronto. É um padrão, mas não tem o formato exigido e as telas já foram desenhadas com o nosso.

8. Códigos de erro em um enum

Por quê: o código é estável e a interface escolhe a mensagem por ele (por exemplo, PONTO_COM_SOLICITACOES_ABERTAS). Cada código já carrega seu status HTTP, então o Service só lança ErroNegocioException(CODIGO, mensagem). Os códigos de regras futuras já estão no enum para os módulos não inventarem nomes diferentes.

Descartado: uma classe de exceção por tipo de erro. Dá mais arquivos e pouco ganho. Também descartado: texto livre como código, que se perde com erro de digitação.

9. Um único handler que estende ResponseEntityExceptionHandler

Por quê: além das nossas exceções, ele padroniza os erros do próprio Spring (JSON malformado, rota inexistente, método não permitido, parâmetro inválido), que de outra forma sairiam em formato diferente.

Descartado: um método para cada exceção sem a classe base, que deixaria esses erros de fora e quebraria o formato uniforme.

10. Erro inesperado: 500 genérico, detalhe só no log

Por quê: RNF03 e RNF06 proíbem expor stack trace, SQL e credenciais. Violação de integridade do banco vira 409 REGRA_NEGOCIO sem a mensagem do driver, que traz nome de tabela e de constraint.

Descartado: devolver a mensagem da exceção "para ajudar no desenvolvimento", que vaza informação e acabaria indo para a entrega.

11. Mensagens de validação em português (locale fixo pt_BR)

Por quê: duas linhas no application.properties traduzem todas as mensagens padrão do Bean Validation, sem escrever mensagem em cada anotação e sem depender do idioma do computador de quem roda.

Descartado: escrever message= em todas as anotações de todos os DTOs, muito trabalho repetido. Esse caminho continua permitido para mensagens específicas.

Paginação

12. page e size inválidos retornam 400 (sem corrigir)

Por quê: é o critério de aceite da issue. A classe Paginacao valida page ≥ 0 e size de 1 a 100 (padrão 20) e aponta o campo no erro.

Descartado: o Pageable automático do Spring, que ajusta em silêncio um size acima do limite em vez de responder 400.

13. Ordem fixa por listagem, com ID como desempate

Por quê: garante ordenação estável (a mesma página sempre devolve os mesmos registros) e impede o cliente de ordenar por campo interno, o que poderia expor dados ou gerar erro 500. Nenhuma tela pediu ordenação escolhida pelo usuário.

Descartado: parâmetro sort livre (inseguro) e sort com lista de campos permitidos (seguro, mas é trabalho a mais sem necessidade agora; pode ser adicionado depois sem quebrar o contrato).

14. Envelope próprio PaginaResposta

Por quê: content, page, size, totalElements e totalPages são os campos do contrato e não mudam se o Spring Data mudar. Os módulos usam PaginaResposta.de(page, conversor), que também converte entidade em DTO e evita expor a entidade (RNF06).

Descartado: devolver o Page do Spring direto, que traz campos extras (pageable, sort, first, last) e expõe a estrutura interna.

DTOs e documentação

15. DTOs como record com Bean Validation

Por quê: menos código que classe com getters, imutável, e a validação (@NotBlank, @Positive, @Valid em listas) funciona igual. Seguimos o que já decidimos na API-01: o DTO fica junto da entidade no módulo, e controllers nunca devolvem entidade JPA nem dados pessoais em endpoints públicos.

Descartado: classes com Lombok, que funcionam, mas deixam o DTO mais longo sem ganho.

16. Documentação por OpenAPI (springdoc)

Por quê: a dependência já está na base da API-01 e gera /v3/api-docs e o Swagger UI a partir dos controllers, sem manter um arquivo à mão. O schema de erro (ErroResposta) e o de página já têm exemplos; cada módulo acrescenta os seus.

Descartado: coleção Postman escrita à mão, que sai de sincronia com o código a cada mudança.

Onde fica no projeto

Tudo em src/main/java/br/edu/ifs/web1/coletaseletiva/shared/: erro/ (CodigoErro, ErroCampo, ErroResposta, as três exceções e o GlobalExceptionHandler), paginacao/ (Paginacao e PaginaResposta) e config/ (OpenApiConfig). Os testes estão em src/test/java/.../shared/.

Validação

Os testes usam um controller que existe só no teste, sem banco, e cobrem entrada inválida, mensagens em português, JSON malformado, registro ausente, rota inexistente, método não permitido, duplicidade, versão, integridade sem SQL, erro 500 sem detalhes, size 0, 101 e negativo, page negativo, padrão 20 e metadados da página.

Comandos: ./mvnw test e, para a documentação, subir a API e abrir /swagger-ui.html.

Resultado real: ./mvnw test -Dtest='GlobalExceptionHandlerTest,PaginacaoTest' em 06/11/2026, Fedora 44, Java 21.0.13 (Temurin): BUILD SUCCESS, Tests run: 22, Failures: 0, Errors: 0.